

ENCODE X ARC PROGRAMMABLE MONEY HACKATHON - DEFI TRACK - CHECKPOINT 2

Standing Orders

Treasury rules in plain English, executed as deterministic onchain flows on Arc.

"route 10% of every incoming payment to savings" - typed once, enforced forever.

AI compiles the sentence. The chain runs the rule. AI touches money zero times.

The problem

- Onchain treasuries are managed by hand: someone watches balances, splits revenue, tops up chains.
- Automation today means either writing custom contracts (slow, risky) or trusting an AI agent with execution
 - unauditable, because the same prompt can produce different transfers tomorrow.
- Finance teams already think in standing orders: fixed instructions, executed exactly. Crypto treasuries deserve the same guarantee.

How it works: sentence to template, then AI leaves

1. **Compile (once, with AI):** one constrained LLM call maps the sentence to one of exactly three fixed templates - Split, Sweep, Bridge - and fills the parameters. Ambiguity = refusal, never a guess.
 2. **Deploy:** parameters land in a thin Blockscout-verified Solidity template on Arc; `execute()` is permissionless.
 3. **Run (forever, no AI):** a keeper watches incoming USDC and schedules, then calls `execute()`. Same event in, same transfer out, every time.
- The rule survives even if our backend dies: anyone can call `execute()`.

Differentiation

	arc-fintech (Circle sample)	ArcFlux (prior hackathon)	Standing Orders
Standing rules	none - manual buttons	scheduled push payments	reactive treasury policy on incoming funds
AI's role	none	GPT-4 parses every command; Guardian Agent scores every transaction	compiled out after setup - zero AI at execution
Onchain contracts	none	none (custodial wallets + Python backend)	verified templates, permissionless execute()
Cross-chain	bridging UI	explicitly none	CCTP Bridge template, Arc to Base

Tech stack

- **Arc Testnet** (chain 5042002): USDC-native gas, sub-second deterministic finality
- **USDC**: treasury asset; we handle Arc's dual native (18d) / ERC-20 (6d) interfaces correctly
- **CCTP V2**: Arc (domain 26) to Base Sepolia (domain 6), attestation via Circle Iris API
- **App Kits**: `@circle-fin/app-kit` - Send (payouts) and Bridge (one-call burn-attest-mint)
- **EIP-7708 watcher**: Arc's system emitter logs EVERY USDC movement - one filter catches native and ERC-20 incoming payments (an Arc-only capability)
- Foundry (contracts), Node + viem (keeper), TypeScript

Status and plan to final submission

- **Done (week 1):** feasibility research fully verified against Circle/Arc primary docs - CCTP lane, App Kits, EIP-7708 watching, Foundry deployment; architecture locked; differentiation verified against arc-fintech and ArcFlux repos.
- **Day 1-2:** spike - prove Arc to Base Sepolia bridge + system-emitter watching end to end
- **Day 3-5:** Split/Sweep/Bridge templates + keeper + interpreter
- **Day 6-9:** dashboard, demo scenarios (including the refusal scenario), polish
- **Aug 9:** submit one day early - video, deck, deployed MVP

Why Arc makes this possible

- On every other EVM chain, native-token payments leave **no logs** - a splitter must poll balances or trace blocks.
- Arc emits an ERC-20-style `Transfer` log from the system emitter `0xffff...fffe` for **every** USDC movement, native or ERC-20 (EIP-7708).
- One log filter = a complete, deterministic feed of incoming treasury payments.
- That single property is what lets a fixed onchain rule react to real-world revenue with no oracle, no indexer, and no AI.